

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ



МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

*Факультет Информационных технологий
Кафедра Информатики и информационных технологий*

направление подготовки

09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 8

Дисциплина: BackEnd-разработка

Тема: Добавление в веб-приложение на основе ASP.NET Core статических файлов

Выполнил: студент группы 241 - 333

Конев Максим Дмитриевич
(Фамилия И.О.)

Дата, подпись _____
(Дата) (Подпись)

Проверил: _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____
(Дата) (Подпись)

Замечания: _____

Москва

2026

Цель работы

Изучить добавление, организацию и обслуживание статических файлов в веб-приложении на платформе ASP.NET Core.

Примечание

Данная лабораторная работа является расширенной версией предыдущей лабораторной работы, в следствие чего могут повторяться некоторые её отдельные части.

Используемые технологии и инструментальные средства

В ходе выполнения работы использовались:

- ASP.NET Core Web API — платформа для разработки веб-приложений
- C# — язык программирования
- Visual Studio — среда разработки
- Middleware — механизм обработки HTTP-запросов
- Static Files Middleware — обслуживание статических ресурсов

Ход выполнения работы

1. Создание проекта

Был создан проект ASP.NET Core Web API. В проекте присутствует стандартная структура, включая папку `wwwroot` для хранения статических файлов.

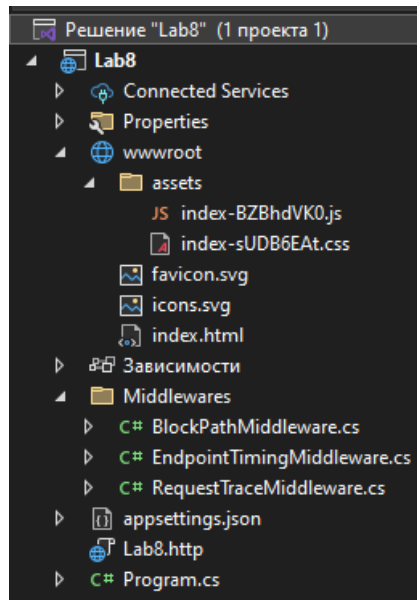


Рисунок 1. Структура проекта

2. Добавление статических файлов

В папку `wwwroot` были добавлены файлы различных типов:

- HTML-файл: `index.html`
- CSS-файл: `index.css`
- JavaScript-файл: `index.js`
- SVG-изображения: `favicon.svg`, `icons.svg`

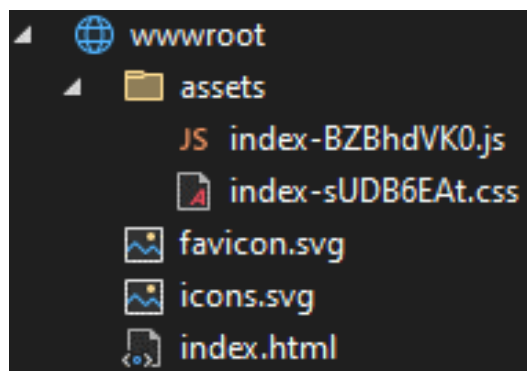


Рисунок 2. Статические файлы проекта

3. Организация структуры каталогов

Для удобства хранения ресурсов была создана вложенная структура для логического разделения ресурсов и упрощения их использования:

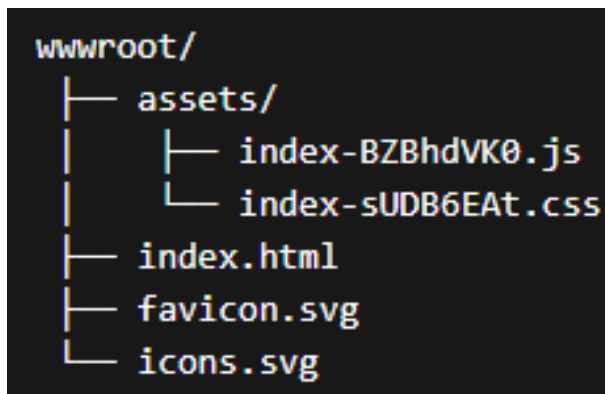


Рисунок 3. Структура каталогов проекта

4. Настройка обслуживания статических файлов

Для включения обработки статических файлов в ASP.NET Core был добавлен middleware:

```
app.UseStaticFiles();
```

Рисунок 4. Middleware для обработки статических файлов

Важно, что подключение выполнено до пользовательских middleware, чтобы статические файлы обрабатывались быстрее.

5. Подключение middleware

Middleware были подключены в строго заданном порядке, так как порядок подключения middleware напрямую влияет на обработку HTTP-запроса. Каждый компонент конвейера выполняется последовательно до вызова

next(), а затем в обратном порядке после его завершения. Таким образом, middleware образуют цепочку обработки, где:

- код до await next() выполняется при входе запроса,
- код после — при формировании ответа.

Неправильный порядок может привести к некорректной работе приложения (например, заголовки не будут добавлены или запрос будет обработан не полностью).

```
// Подключение статических файлов
app.UseStaticFiles();

// Подключение middleware в правильном порядке
app.UseMiddleware<BlockPathMiddleware>();
app.UseMiddleware<RequestTraceMiddleware>();
app.UseMiddleware<EndpointTimingMiddleware>();
```

Рисунок 5. Подключение Middleware

6. Реализация endpoint'ов

Были реализованы три минимальных endpoint'а:

1. GET /ping, возвращающий строку pong.
2. GET /trace, возвращающий TraceId, сохранённый в HttpContext.Items.
3. GET /error, выбрасывающий исключение для тестирования обработки ошибок.

```

app.MapGet("/ping", () =>
{
    return "pong";
});

app.MapGet("/trace", (HttpContext context) =>
{
    return Results.Ok(context.Items["TraceId"]);
});

app.MapGet("/error", () =>
{
    throw new Exception("Error for handling");
});

```

Рисунок 6. Реализация endpoint'ов

7. Проверка доступа к статическим файлам

Проверка выполнялась через браузер:

URL	Результат
/index.html	Html-страница
/assets/index-sUDB6EAt.css	CSS файл
/assets/index-BZBhdVK0.js	JS файл
/favicon.svg	изображение

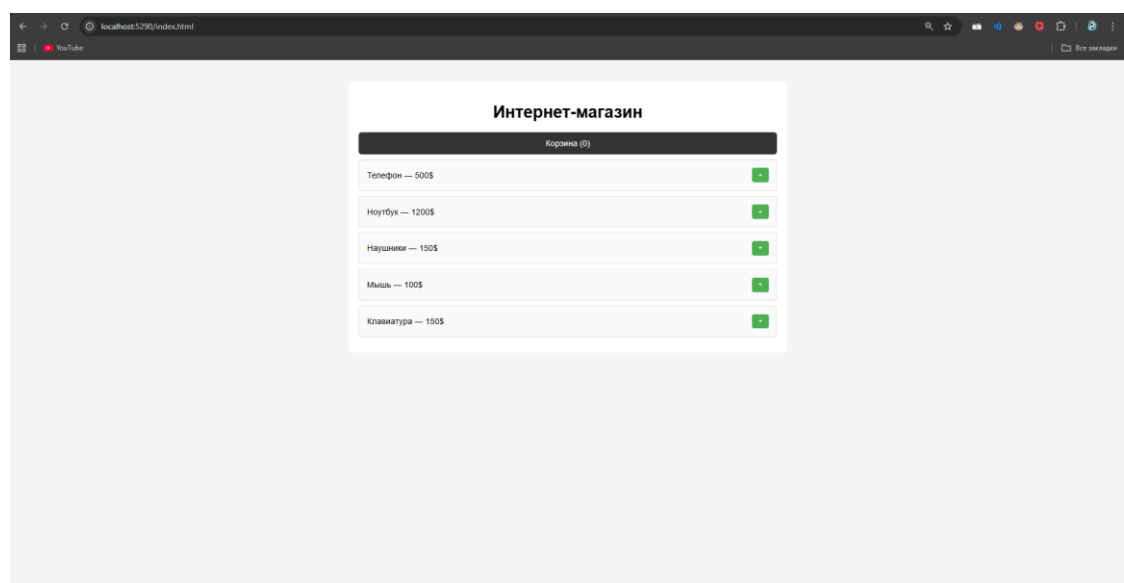


Рисунок 7. Открытие /index.html

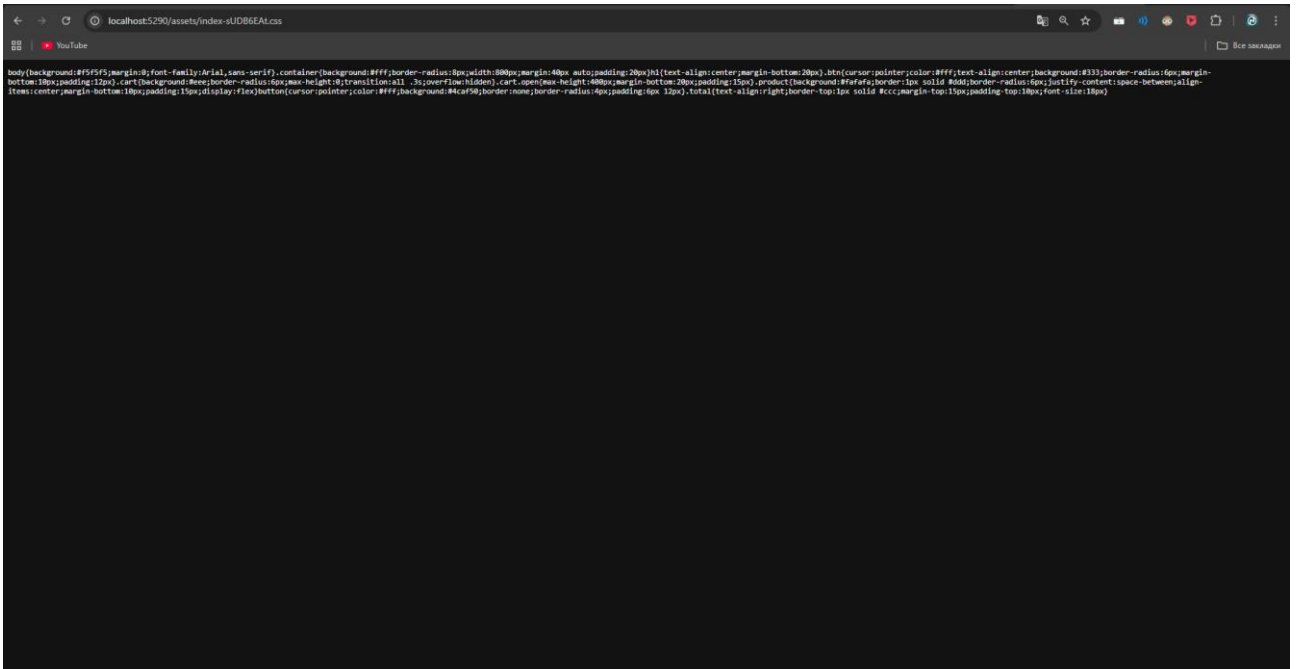


Рисунок 8. Открытие /assets/index-sUDB6EAt.css



Рисунок 9. Открытие /assets/index-BZBhdVK0.js



Рисунок 10. Открытие /favicon.svg

8. Возможные сложности

В процессе выполнения могли возникнуть следующие сложности:

- Понимание роли wwwroot
- Правильный порядок подключения middleware
- Отсутствие доступа к файлам без UseStaticFiles()
- Ошибка при загрузке index.html без UseDefaultFiles()
- Неправильные пути к ресурсам в HTML

Вывод

В ходе выполнения лабораторной работы были изучены механизмы работы со статическими файлами в ASP.NET Core. Были добавлены и структурированы ресурсы различных типов, настроено их обслуживание через middleware, а также проверена корректность доступа через браузер. Практическая часть позволила закрепить понимание работы wwwroot, порядка middleware и принципов обработки HTTP-запросов в ASP.NET Core.

Листинг кода

Листинг 1. Файл Program.cs:

```
using Lab8.Middlewarees;

var builder = WebApplication.CreateBuilder(args);

// Регистрация middleware в DI-контейнере
builder.Services.AddTransient<BlockPathMiddleware>();
builder.Services.AddTransient<RequestTraceMiddleware>();
builder.Services.AddTransient<EndpointTimingMiddleware>();

var app = builder.Build();

// Поключение статических файлов
app.UseStaticFiles();

// Подключение middleware в правильном порядке
app.UseMiddleware<BlockPathMiddleware>();
app.UseMiddleware<RequestTraceMiddleware>();
app.UseMiddleware<EndpointTimingMiddleware>();

// Endpoint для проверки работы
app.MapGet("/ping", () => "pong");

// Возврат TraceId из HttpContext
app.MapGet("/trace", (HttpContext context) =>
{
    return Results.Ok(context.Items["TraceId"]);
});

// Генерация ошибки
app.MapGet("/error", () =>
{
    throw new Exception("Error for handling");
});

app.Run();
```

Листинг 2. Файл BlockPathMiddleware.cs:

```
namespace Lab7.Middlewarees
{
    public class BlockPathMiddleware : IMiddleware
    {
        public async Task InvokeAsync(HttpContext context, RequestDelegate next)
        {
            var path = context.Request.Path;

            // Проверка пути запроса
            if (path.StartsWithSegments("/blocked"))
            {
                // Возврат 403 без передачи дальше
                context.Response.StatusCode = StatusCodes.Status403Forbidden;
                return;
            }
            else
            {
                // Передача запроса дальше
                await next(context);
            }
        }
    }
}
```

```

    }
}

```

Листинг 3. Файл RequestTraceMiddleware.cs:

```

namespace Lab7.Middlewares
{
    public class RequestTraceMiddleware : IMiddleware
    {
        public async Task InvokeAsync(HttpContext context, RequestDelegate next)
        {
            // Генерация уникального идентификатора запроса
            var traceId = Guid.NewGuid().ToString();

            // Добавление в заголовки ответа
            context.Response.Headers.Append("X-Trace-Id", traceId);

            // Сохранение в HttpContext
            context.Items["TraceId"] = traceId;

            await next(context);
        }
    }
}

```

Листинг 4. Файл EndpointTimingMiddleware.cs:

```

using System.Diagnostics;
namespace Lab7.Middlewares
{
    public class EndpointTimingMiddleware : IMiddleware
    {
        public async Task InvokeAsync(HttpContext context, RequestDelegate next)
        {
            var sw = Stopwatch.StartNew();

            // Добавление времени выполнения в заголовок
            context.Response.OnStarting(() =>
            {
                sw.Stop();
                context.Response.Headers.Append("X-Endpoint-Elapsed-Ms",
                    ((long)sw.Elapsed.TotalMilliseconds).ToString());

                return Task.CompletedTask;
            });

            // Выполнение следующего middleware / endpoint
            await next(context);
        }
    }
}

```